Flow Control and Congestion Control

Lecture 19

Computer Networks and Distributed Programming

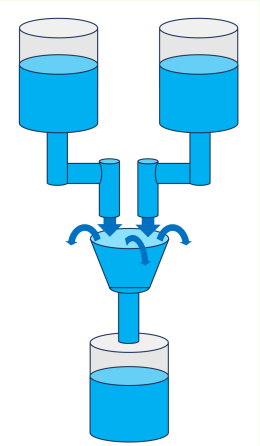
Where we are: two ways to send too fast

There is one field of the header we have never used, and one limit that is in no header at all.

- The receive window says how much room is left in the buffer at the other end: a limit set by the receiver.
- The other limit is set by the network, which announces nothing and has to be guessed from what comes back.
- Both end in the same action — send less — for reasons that have nothing to do with each other.
- Reliability told us what to do *after* a loss. Nothing yet says how not to cause one.
- And both end in the same place: the moment your `write` returns, or does not.

Buffers overflow at both ends

- Loss on the Internet is rarely a broken wire: it is almost always a buffer that was full.
- At the receiver, if the application is not reading fast enough for what is arriving.
- At the routers, if more arrives than the outgoing link can carry away.
- Back to the water: the buffers are buckets along the way, and a bucket that overflows spills.
- Retransmission and timeouts treat the symptom — and add traffic to a network that already had too much.



Flow control is speed matching

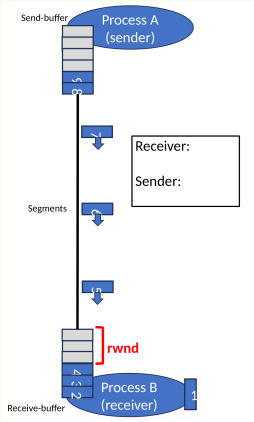
- Correct, in-sequence bytes go into the receive buffer. The application reads them *when it wants*, not when they arrive.
- It may be busy with something else for a long time, and the bytes keep coming in the meantime.
- Consequence: a fast sender and a slow reader overflow the buffer, with a network that did everything right.
- Flow control: a speed-matching service: it matches the rate of the sender to the rate of the *reading application*.
- Congestion control throttles the sender too, for a completely different reason. The two are confused constantly.
- UDP has none of this: its buffer in front of the socket fills up and datagrams are dropped without a word.

The receive window

- RcvBuffer: the size of the receive buffer, in bytes.
- LastByteRead: the last byte the application has taken out; LastByteRcvd: the last byte that has arrived.
- Assume out-of-order segments are discarded, so what is in the buffer is one contiguous run.
- Receive window: the room that is free:

  $$\texttt{rwnd} = \texttt{RcvBuffer} - (\texttt{LastByteRcvd} - \texttt{LastByteRead})$$

- It changes every time either number moves — so it has to be announced again and again.



What the sender promises

- The receiver puts its current `rwnd` in the receive window field of every segment it sends back.
- The sender keeps two numbers of its own: `LastByteSent` and `LastByteAcked`.
- Their difference is the data in flight — sent, and not yet acknowledged by anybody.
- The rule: $\texttt{LastByteSent} - \texttt{LastByteAcked} \leq \texttt{rwnd}$
- Keep to it and the receive buffer cannot overflow: the sender never has more outstanding than there is room for.
- The connection is full duplex, so each direction has its own window and each end announces one.

When the window closes

B advertises `rwnd = 0` and has nothing of its own to send. The application at B then empties the buffer. Who speaks first?

Nobody. TCP sends a segment only when it has data or an acknowledgement to send, so the new window is never announced — and A, which may send nothing, waits forever.

- The specification breaks the deadlock from the sender's side: A keeps sending segments with one byte of data.
- The receiver acknowledges them, and sooner or later one of those acknowledgements carries a non-zero window.
- These are the window probes, and you will see them in a capture: a stalled transfer is not a silent one.

Sixteen bits are not enough any more

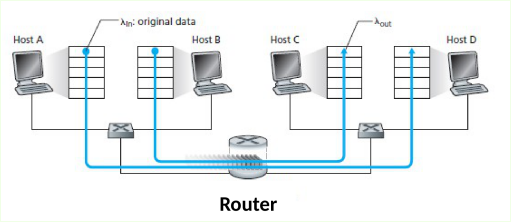
- The field holds at most 65 535: the largest window a receiver could ever announce is 64 KB.
- A window is also a speed limit — at most one windowful per round trip, so at most $\texttt{window}/RTT$.
- 64 KB over a 100 ms path is about 5 Mbit/s, whatever the link underneath is worth.
- Bandwidth-delay product: 1 Gbit/s over that same path is 12.5 MB in flight: two hundred times the field.
- So the window scale option carries a shift count, agreed in the two SYN segments and never again.
- The number in the field is not the window: it has to be shifted. A capture that missed the handshake cannot know by how much.

A different reason to slow down

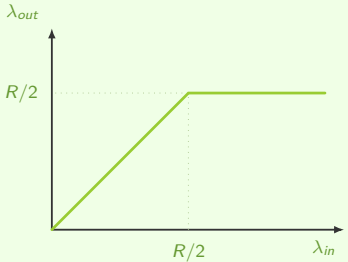
- Flow control is an agreement between the two ends. The network is not party to it and does not appear in it.
- But the path is shared: a sender can be well inside the receiver's window and still be putting in more than the path carries.
- Nobody says so. Routers send no complaint, and neither end can see the queue that is filling up.
- Two questions: what does congestion actually cost, and how can a sender find out that it is happening?
- The first is answered by three scenarios, and the answer is worse each time.

Two senders, one router, infinite buffers

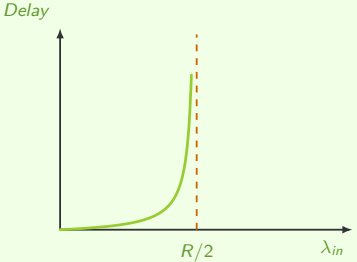
- Two connections share one outgoing link of capacity R. Each application sends at λ_{in} bytes/s.
- No retransmission, no flow control, no congestion control: the transport protocol here does nothing at all.
- Routers are store-and-forward: they hold a packet before passing it on, so they must have buffers.
- In this first scenario those buffers are infinite, so nothing is ever dropped.



Throughput stops, delay does not



a. everything sent arrives — and never more than $R/2$

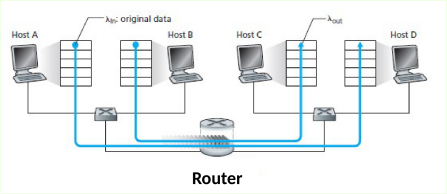


b. and it waits longer and longer to do it

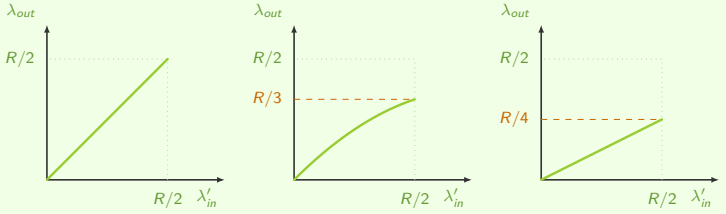
Per connection the throughput follows the sending rate up to $R/2$ — the two split the link — and stays there however hard the hosts push. The right-hand graph is the cost: the delay grows without bound as λ_{in} approaches $R/2$. Nothing is lost here. Queues alone, with no loss at all, already make a link near capacity unusable.

Finite buffers: three rates instead of one

- Buffers are finite, so a packet arriving to a full one is dropped. And now the connection is reliable: what is dropped is sent again.
- λ_{in}: original data, what the application hands to the socket.
- λ'_{in}: the offered load: original data *plus* retransmissions, which is what the link actually sees.
- λ_{out}: what is delivered at the far end.
- In the first scenario the three were the same number. They are not any more, and the gaps between them are the cost.



Three degrees of wasted work



a. no congestion at all — *b. one copy of what was lost* — *c. copies of what had already arrived*

$R/3$ and $R/4$ are example values: what they show is the degradation, not a constant

(a) A sender that magically knew when a buffer was free: nothing is lost, and what is offered is delivered.

(b) Retransmitting only what was truly lost: at an offered load of $R/2$ only $R/3$ is useful data — the rest of the link carries second copies.

(c) Timers that fire too early: each packet is forwarded twice on average, and the useful share falls to $R/4$.

When the loss happens on the second hop

- Four senders now, on paths of two hops that overlap: each connection competes with a different one at each router.
- Traffic that has already crossed the first router has to fight for buffer space at the second one.
- As the offered load grows, the throughput of a connection does not level off — it falls, and in the limit goes to zero.
- The worst cost: when a packet is dropped at the second hop, all the capacity spent carrying it that far was wasted.
- The first router would have done better staying idle. This is congestion collapse, and it happened to the real Internet.
- Everything that follows was written in answer to it.

Who tells the sender

- End-to-end: the network says nothing, and congestion has to be inferred from what the end system can observe.
- Loss and delay are all there is. This is what classic TCP does, and it is what the IP layer forces: IP offers no feedback.
- Network-assisted: routers say so explicitly, and the sender is told rather than left to guess.
- The message can go straight back to the sender — a choke packet — or forward as a mark on a packet, which the receiver then echoes.
- The second way takes a full round trip, and is the one the Internet actually adopted.

ECN: two bits and two flags

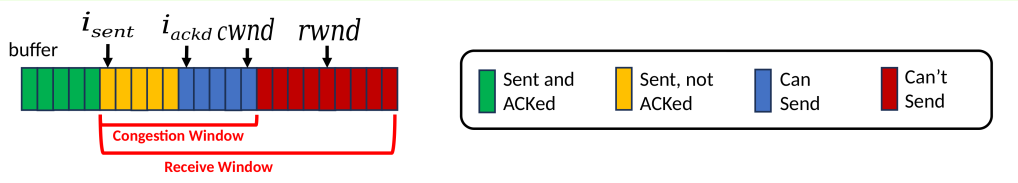
- Two bits in the IP header carry it: one setting says both ends can handle ECN, another says *this router is congested*.
- The congested router does not drop anything — it marks a datagram that is already on its way to the receiver.
- The receiving TCP turns the mark round and sets ECE in an acknowledgement going back to the sender.
- The sender halves its window as if a segment had been lost, and sets CWR on its next segment to say it has done so.
- The point: the signal can arrive *before* the loss, so the cost of the previous three frames is never paid.
- It only works if the two ends and the routers in between all agree to use it.

Three questions

TCP takes the end-to-end road, so each sender has to work it out alone.

- Rate regulation: by what means does a sender limit the rate at which it puts traffic into the connection?
- Congestion detection: how does it find out that the path between itself and the destination is congested?
- Rate adjustment: by what rule does it change the rate once it has an opinion about the path?
- And the constraint behind all three: senders are not coordinated. Each acts on local information, asynchronously.

The second limit on the sender



- The sender keeps one more variable, the congestion window `cwnd`. It is in no header: the receiver never learns it.
- Read the buffer left to right: green is acknowledged, and the first boundary is the last byte acknowledged. Yellow is in flight, so the next boundary is the last byte sent.
- Blue may go out now, red may not — and which of the two brackets stops it is the whole point: $\texttt{LastByteSent} - \texttt{LastByteAcked} \leq \min\{\texttt{cwnd}, \texttt{rwnd}\}$
- Raising and lowering `cwnd` is how the rate is regulated. The smaller of the two windows always decides.

Rate is a window divided by a round trip

- Roughly: at the start of each round trip the sender may put `cwnd` bytes in, and by the end they have been acknowledged.
- So the sending rate is about $\texttt{cwnd}/RTT$, and changing the window is the only way the sender has to change it.
- The increases are triggered by arriving acknowledgements: TCP is said to be self-clocking.
- A path that returns acknowledgements slowly therefore opens its window slowly, without anyone deciding that it should.
- Consequence: two connections through the same bottleneck with different round trips do not grow at the same speed. It matters later.

What counts as congestion

Loss event

A timeout, or the arrival of three duplicate acknowledgements — four acknowledgements in all for the same byte.

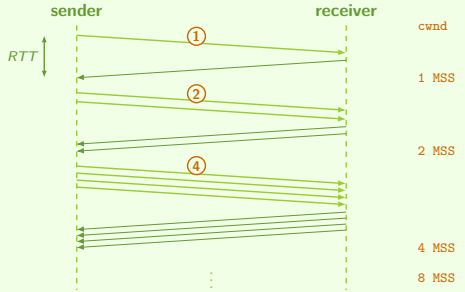
- Both mean a datagram was dropped, and a dropped datagram almost always means a router buffer that was full.
- So the sender reads a loss event as a statement about the path, not about the wire or the receiver.
- No loss events, and acknowledgements arriving as expected, means the path is delivering: the window may grow.
- Notice what is *not* used: no measurement of delay, and no message from any router. Only the two events above.

Probing for bandwidth

- There is no correct rate to compute. The sender increases until it hurts, backs off, and starts increasing again.
- A lost segment: means congestion: lower the rate.
- An acknowledged segment: means the path is delivering: the rate may go up.
- Bandwidth probing: do both, forever. The sender walks up to the point where loss begins, retreats, and walks up again.
- The algorithm is Jacobson's, and it has three parts: slow start, congestion avoidance, fast recovery.
- The first two are mandatory; the third is recommended, and is what separates the two versions of TCP everyone quotes.

Slow start

- `cwnd` starts at 1 MSS: with a 500-byte MSS and a 200 ms path, about 20 kbit/s. Far too little, and the sender has no idea by how much.
- It grows by one MSS for every segment acknowledged, and a round trip brings back as many acknowledgements as there were segments.
- So the window doubles every round trip: 1, 2, 4, 8. Linear in acknowledgements, exponential in time — what is slow is the starting point, not the growth.



one more MSS for every segment acknowledged — and in one round trip there are as many acknowledgements as there were segments

Where slow start stops

Doubling cannot go on. It ends in exactly three ways.

- A timeout: `ssthresh` is set to cwnd/2, `cwnd` goes back to 1 MSS, and slow start begins again.
- cwnd reaches ssthresh: doubling from here would be reckless, so TCP changes to congestion avoidance.
- Three duplicate acknowledgements: fast retransmit, and then fast recovery.
- ssthresh: the memory of where it hurt last time. It starts large — 64 KB — because at the beginning there is no memory at all.
- Every loss event writes it again, always as half the window that was in use when the loss was detected.

Congestion avoidance: one MSS per round trip

- On entry `cwnd` is about half of what it was when the loss happened. Trouble is close, so the caution is deliberate.
- Instead of doubling, the window grows by one MSS per round trip. Additive, where slow start was exponential.
- Per acknowledgement that means a fraction: $\texttt{cwnd} \mathrel{+}= \texttt{MSS} \cdot (\texttt{MSS}/\texttt{cwnd})$.
- Check it: with $\texttt{cwnd} = 10$ MSS, ten acknowledgements arrive in a round trip and each adds a tenth: one MSS in all.
- The larger the window, the smaller each step — which is what keeps the increase linear in time instead of exponential.

Two losses, two reactions

Both are loss events. Why does a timeout send `cwnd` back to 1 MSS, while three duplicate acknowledgements only halve it?

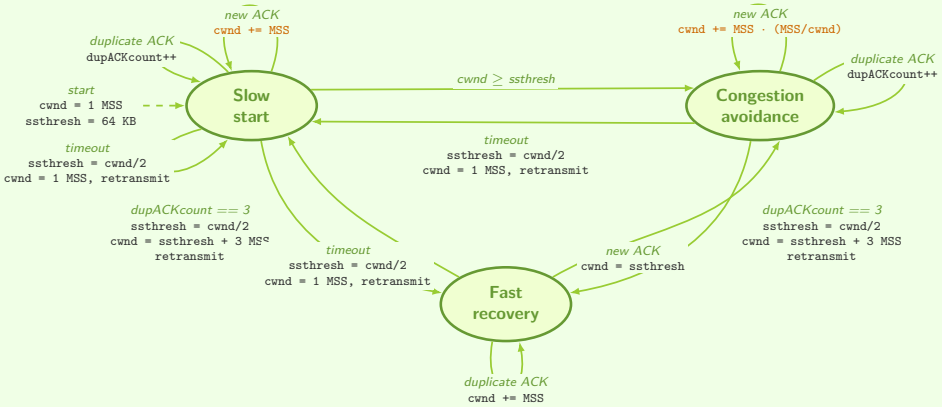
Because three duplicates are proof that the network is still delivering: segments after the missing one got through, and something came back to say so. A timeout is silence, and silence can mean anything.

- So the timeout reaction is the drastic one: half the window remembered in `ssthresh`, the window itself back to 1 MSS.
- The triple-duplicate reaction keeps half the window and carries on from there.
- TCP Tahoe did not make this distinction — any loss meant starting over. Fast recovery is what was added.

Fast recovery

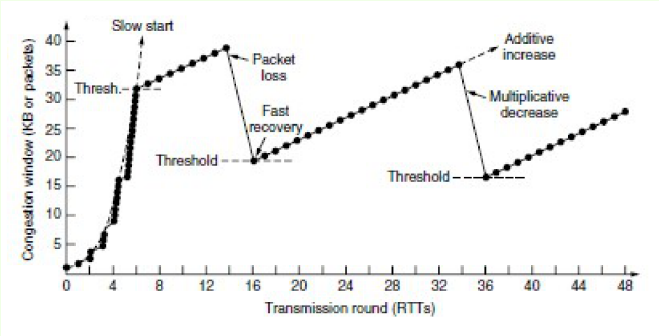
- Entered on the third duplicate: `ssthresh` becomes cwnd/2, `cwnd` becomes `ssthresh` + 3 MSS, and the missing segment goes out at once.
- The +3 is not decoration: three duplicates mean three segments have left the network and arrived.
- Each further duplicate adds one more MSS, for the same reason — the pipe is emptying while the sender waits.
- When the acknowledgement for the missing segment finally arrives, `cwnd` drops to `ssthresh` and congestion avoidance resumes.
- A timeout during recovery abandons all of it: back to 1 MSS and slow start.

The whole algorithm, in one machine



Three states, two events that matter. The two growth rules are what separates the states on the top row; going down is the mild loss, coming back up is the severe one.

Tahoe, Reno, and the sawtooth



- Exponential rise, linear climb, loss — and the threshold halved and written down again.
- Tahoe: any loss goes back to 1 MSS. Reno: a triple duplicate halves instead — which is the tooth you see.
- Ignore the start and TCP is additive increase, multiplicative decrease; a tooth averages about $0.75 \cdot W/RTT$.

What your machine actually runs

- Classic Reno is the model, not the code. It is what is worth understanding, and it is not what your kernel is running.
- CUBIC: same slow start, same fast recovery, different congestion avoidance: the window follows a cubic function of the time since the last loss.
- It climbs back quickly to the window it had before the loss, then probes cautiously around it — and is the Linux default.
- Delay-based: Vegas and BBR watch the round trip *growing* and slow down before anything is lost.
- Keep the pipe just full, but no fuller: a full pipe is useful work, a full queue is only delay.
- For the capture at the end: the sawtooth you find will probably not be Reno's, and now you know why.

Why two connections converge to the same share

- K connections through one bottleneck of rate R: the mechanism is fair if each of them ends up with about R/K.
- Take two, with the same MSS and the same round trip. While nothing is lost, both add one MSS per round trip.
- Adding the *same* amount to both does not change the difference between them: the pair moves along a line at 45 degrees.
- Losing halves *both*, and halving two unequal numbers brings them closer together — proportionally, the larger one loses more.
- Repeat, and the pair converges on the equal share whatever it started from.
- The reason: the decrease is multiplicative and the increase is not. Fairness is a consequence of that asymmetry.

And why fairness is easy to break

- The argument assumed equal round trips. A shorter path opens its window faster and takes more of the link, for free.
- UDP does none of this: it has no window, no loss events and no reason to slow down. It simply keeps sending.
- Your own chat over datagrams retransmits what was lost and never once reduces its rate: it is exactly the unfair flow of this picture.
- Applications choose UDP for that reason, and a TCP connection sharing a link with them gets what is left.
- Nine applications with one connection each and a tenth with eleven: the tenth takes more than half the link, all of it legally.
- So: the fairness of TCP is a convention among senders, not a rule the network enforces.

What is under your write

`write` does not transmit — we said so when the two buffers appeared, and left the rest open. This is the rest.

- It copies bytes into the send buffer and returns. TCP takes them out later, on its own schedule.
- How many may leave right now: $\min\{\texttt{cwnd}, \texttt{rwnd}\}$, less whatever is already in flight.
- So the buffer drains at the speed the peer and the path allow, and the application never sees either number.
- A blocking write returns when there is room: — so it blocks when the buffer is full, which means both limits have been reached.
- A short write is the same event, halfway: the buffer took part of what you offered and filled up. That is why you loop.
- On short messages neither ever happens, which is why both are discovered late.

The client that stopped reading

One client in your chat stops calling `read` — it is not dead, just busy. What reaches your server?

Its receive buffer fills, so it advertises rwnd $= 0$. The sender's own buffer then fills behind it, and the `write` in your worker blocks — holding whatever lock it was holding.

- Nothing is broken and nothing is lost. This is flow control working exactly as designed, seen from the wrong end.
- On the wire it is visible: a window shrinking to zero, then the one-byte probes, then a window update when the client reads again.
- The bug was never the slow client. It was blocking while holding the roster lock, which turned one silent client into a frozen chat.

EAGAIN, and what it does not tell you

```c
ssize_t n = write(fd, out[i].buf, out[i].len);

if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
    FD_SET(fd, &wall);      /* send buffer full: min(cwnd,rwnd) is spent */
    return;                 /* come back when it drains; nobody waits */
}
if (n < 0 && errno == EINTR) return;       /* a signal, not a stall */
if (n < 0) { drop(i); return; }

out[i].len -= n;            /* a short write lands here too, and is normal */
memmove(out[i].buf, out[i].buf + n, out[i].len);
```

EAGAIN says the send buffer is full *now*, and nothing about why. rwnd travels in every segment and you can read it in a capture; cwnd never leaves the sender's kernel. Two causes, one symptom, and only one of them visible from outside.

Summary

- Flow control protects the receiver's buffer. `rwnd` is the free room, it travels in every segment, and zero is a legal value.
- Congestion control protects the network, which asks for nothing and reports nothing: the sender has to infer it.
- Congestion costs delay before it costs loss, and a packet dropped late wastes every link that already carried it.
- `cwnd` is the sender's own limit, in no header; a loss event is a timeout or three duplicate acknowledgements.
- Slow start doubles, congestion avoidance adds one MSS per round trip, fast recovery halves instead of starting over.
- And your `write` blocks when $\min\{\texttt{cwnd}, \texttt{rwnd}\}$ has nothing left to give. Next: the routers whose queues we have been talking about all along.

Capture a ten-megabyte download

Start the capture first, on the interface you are really using. Then:

```
$ wget http://speedtest.tele2.net/10MB.zip
```

- One long transfer in one direction is the easiest thing there is to read: a single flow, no interleaving, and a known size.
- Right-click any segment of it, then Follow → TCP Stream: Wireshark writes the filter `tcp.stream eq N` for you and everything else disappears.

(a) Which four numbers is that filter standing for, and which one of them did nobody choose?

(b) `wget` took the server's port from the URL. Could you have chosen your own local port as well?

The sequence-number graph

Statistics → TCP Stream Graphs → Time/Sequence (tcptrace), with the direction that carries the data.

- Time along the bottom, sequence number up the side — so the slope of the curve is the throughput.
- The lower line is the data actually sent. The upper one is the highest sequence number the receiver has allowed: acknowledgement plus window.
- The vertical distance between them is what is in flight, and it is the closest thing to a picture of the window you will get.
- The first seconds bend *upwards*, then the curve straightens: that bend is the window opening, and the straight part is the steady state.

(c) Find where the bending stops. Roughly how many round trips did it take, and what does that make the path's RTT?

The window you can see, and the one you cannot

```
tcp.analysis.bytes_in_flight      unacknowledged bytes, packet by packet
tcp.analysis.duplicate_ack        the symptom of a loss
tcp.analysis.window_update        the receiver opening up again
tcp.analysis.zero_window          the receiver with nothing left
```

- Combine them with `&&` in the display filter bar of the main window.
- `bytes_in_flight` is Wireshark's own arithmetic on what it saw. It equals `cwnd` only when the sender always has data and `rwnd` is not the smaller window. Otherwise it is a lower bound.

(d) Look at the window scale factor in the two SYN segments. What is the largest window either side could announce?

What to look for

State	What shows up in the capture
Slow start	the curve bending upwards, few segments per round
Congestion avoidance	a straight line, a steady rate of segments
Fast recovery	duplicate acknowledgements, one retransmission, a step

(e) Find a retransmission. Was it preceded by three duplicate acknowledgements, or by a silence? Which of the two reactions should follow?

(f) Capture again with `wget --limit-rate=200k`. Which of the two windows runs out this time, and which one never does?